

Problem 17 – Smart School Attendance Manager

Difficulty: ★★☆☆ Intermediate+

Platform:  Streamlit Only

Main New Skills:  Historical Data +  Indexing +  Slicing

Overview

A school wants to replace its paper attendance sheets with a digital attendance management system.

Build a **Smart School Attendance Manager** that allows teachers to:

-  Register students
-  Record attendance for different dates
-  Search for students
-  View attendance history
-  Correct attendance records
-  Generate attendance statistics
-  Identify students with low attendance

Main Learning Idea

In previous problems, you mainly worked with the **current state** of data.

Now, a student should have an **attendance history containing multiple records**.

For example:

```
attendance_history = [  
    {"date": "2026-10-01", "status": "Present"},  
    {"date": "2026-10-02", "status": "Absent"},  
    {"date": "2026-10-03", "status": "Present"}  
]
```

This problem introduces an important change:

One student → many historical records

Mission — Python

1. Store Student Data

Create a suitable data structure to store student information.

Each student should contain:

- Student ID
- Student name
- Class
- Attendance history

The attendance history must support **multiple dates**.

2. Practice Indexing

Use **indexing** to access individual attendance records.

Your application should demonstrate that you can:

- Access the first attendance record
- Access the last attendance record
- Access a specific record by its position

For example:

`history[0]`

`history[-1]`

Requirement

The application should have a small section such as:

Attendance Record Preview

where the user selects a student and the application displays:

- First recorded attendance
 - Most recent attendance
-

3. ✂ Practice Slicing

Use **slicing** to extract part of a student's attendance history.

The application should allow the teacher to view:

Recent Records

Display the **last 5 attendance records** using slicing.

Example:

```
recent_records = history[-5:]
```

If the student has fewer than 5 records, display all available records.

★ Important

Do **not** manually loop through the entire history just to obtain the last five records.

The purpose of this requirement is to practice **slicing**.

4. 📄 Register a Student

Create a registration form containing:

- Student ID
- Student name
- Class
- **Register Student** button

Validation

Before saving:

- All required fields must be completed.
 - Student ID must be unique.
 - Display an appropriate message after registration.
-

5. Record Attendance

Create an attendance form.

The teacher should select:

- Student
- Attendance date
- Status

Possible statuses:

-  Present
-  Absent

Rules

- Attendance date cannot be in the future.
 - A student cannot have two attendance records for the same date.
 - Display a warning if attendance has already been recorded.
 - Display a success message after recording.
-

6. Correct Attendance

Allow the teacher to correct an existing attendance record.

The teacher should select:

- Student
- Attendance date
- New status

For example:

Old Status: Absent

New Status: Present

The correction must **update the existing record**.

It must NOT create a second record for the same student and date.

7. 👤 View Students

Display all registered students.

For each student show:

- Student ID
- Name
- Class
- Number of attendance records
- Attendance percentage

If there are no students:

 No students have been registered yet.

8. 🔍 Search Students

Allow searching by:

- Student ID
- Student name
- Class

Search should be case-insensitive where appropriate.

Display all matching students.

If no students are found:

 No matching students found.

9. 📅 Attendance History

Create a section where the teacher selects a student and views their **complete attendance history**.

Display each record with:

- Date
- Status

The records should be displayed in chronological order.

10. 📅 Today's Attendance

Create a report showing today's attendance.

Divide students into:

🟢 Present

Students who are present today.

🔴 Absent

Students who are absent today.

Also display:

- Total students
- Present today
- Absent today
- Today's attendance percentage

If no attendance has been recorded today:

🔴 No attendance has been recorded today.

11. 📊 Attendance Statistics

Create a statistics section.

Display:

Overall Statistics

- 🧑 Total students
- 📋 Total attendance records
- 🟢 Total present records
- 🔴 Total absent records
- 📈 Overall attendance percentage

Student Statistics

For a selected student:

- Total attendance records
- Present records
- Absent records
- Attendance percentage

Class Statistics

Display attendance statistics for each class.

12. Low Attendance Report

Allow the teacher to choose a minimum attendance percentage.

For example:

Minimum Attendance: 75%

Display students whose attendance percentage is below the selected value.

Use a warning message when students are found.

If every student meets the required percentage:

 All students are meeting the attendance requirement!

13. Delete a Student

Allow the teacher to delete a student.

When a student is deleted:

- Remove the student.
 - Remove all attendance history belonging to that student.
 - Display a success message.
-

Streamlit UI Requirements

Page Configuration

Use:

- `st.set_page_config()`
- Wide layout
- An appropriate emoji page icon

Example concept:

 Smart School Attendance Manager

Sidebar

The sidebar should display:

-  School name
 -  Today's date
 -  Number of registered students
 -  Number of attendance records
 -  Present records
 -  Absent records
 -  Overall attendance percentage
-

Tabs

Create four main tabs:

Tab 1 — Students

- Register students
- View students
- Delete students

Tab 2 — Attendance

- Record attendance
- Correct attendance
- View today's attendance

Tab 3 — Search & History

- Search students
- View complete attendance history
- View first/latest records using indexing
- View recent records using slicing

Tab 4 — Reports

- Attendance statistics
 - Class statistics
 - Low-attendance report
-

Layout Requirements

Use:

- `st.sidebar`
- `st.tabs()`
- `st.columns()`
- `st.divider()`

For example, today's attendance can use:

Present		Absent	
Ahmed		Omar	
Sara		Ali	

Forms

Use `st.form()` for:

Student Registration

- Student ID
- Student name
- Class
- Submit button

Attendance Recording

- Student
- Date
- Status
- Submit button

Attendance Correction

- Student
 - Date
 - New status
 - Submit button
-

UI & Formatting Requirements

Your application must use visual formatting intentionally.

Text Colors

Use appropriate colors to communicate meaning:

-  Green → Present / Success
-  Red → Absent / Errors
-  Yellow → Warnings
-  Blue → Information

You may use `st.markdown()` with HTML/CSS where appropriate.

Buttons

Use both:

-  Primary buttons for important actions
-  Secondary buttons for less important actions

Text Alignment

Use text alignment where appropriate.

For example:

 SMART SCHOOL ATTENDANCE MANAGER

The main title or important headings can be centered.

Dividers

Use:

`st.divider()`

to separate major sections.

Metrics

Use:

`st.metric()`

for important statistics such as:

- Total Students
 - Present Today
 - Absent Today
 - Attendance Rate
-

Required Status Elements

Your application must meaningfully use all of these:

- `st.success()`
- `st.info()`
- `st.warning()`
- `st.error()`
- `st.spinner()`

Examples

 `st.success()`

After successfully registering a student.

 `st.info()`

When there are no attendance records.

 `st.warning()`

When attendance already exists for the selected date.

 `st.error()`

When the user enters a duplicate Student ID.

 `st.spinner()`

While processing an attendance operation.

★ Bonus Mission — Attendance History Explorer

Build an **Attendance History Explorer**.

The teacher selects a student and chooses one of the following:

First Record

Display the first attendance record using indexing.

`history[0]`

Latest Record

Display the latest attendance record using negative indexing.

`history[-1]`

Recent Records

Display the latest 5 records using slicing.

`history[-5:]`

Custom History Preview

Allow the teacher to choose how many recent records to display.

For example:

Show recent records: 3

Then display:

`history[-3:]`

The interface should clearly show which operation is being demonstrated:

 Indexing → First Record

 Negative Indexing → Latest Record

 Slicing → Recent Records

Optional Challenge — Attendance Trend Report

Create an attendance trend report for a selected student.

The teacher chooses:

- Student
- Start date
- End date

Display:

- Number of recorded days
- Present days
- Absent days
- Attendance percentage
- Recent attendance records

Use **date filtering + slicing** to create a useful historical report.

Required Streamlit Functions

Use the following:

Text / Display

- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.code()`
- `st.divider()`
- `st.markdown()`

Input

- `st.text_input()`
- `st.number_input()`
- `st.selectbox()`
- `st.radio()`
- `st.date_input()`
- `st.button()`
- `st.form()`
- `st.form_submit_button()`

Layout

- `st.sidebar`
- `st.tabs()`
- `st.columns()`

Status

- `st.success()`
- `st.info()`
- `st.warning()`
- `st.error()`
- `st.spinner()`

State / Display

- `st.session_state`
 - `st.metric()`
-

Python Topics

This problem should reinforce:

- Variables
- Strings
- Integers
- Floats
- Booleans
- Lists
- Dictionaries
- Nested data structures
- Arithmetic operators
- Comparison operators
- Logical operators
- Membership operators
- Assignment operators
- if
- elif
- else
- for loops
- Functions
- datetime
- Dates
- Date comparison
- Searching
- Updating records
- Validation

New Python Skills

Indexing

history[0]

history[-1]

Slicing

history[-5:]

history[:3]

Important distinction

The student should understand:

Indexing gets one item.

Slicing gets multiple items.

Streamlit-Only Rule

This problem must be built entirely with Streamlit.

Do **NOT** use:

-  input()
-  print()
-  Console menus
-  while True
-  Exit options

Streamlit reruns the application when the user interacts with it, so a console-style loop is unnecessary.

Learning Objectives

After completing this project, you should be able to:

- Design a real-world attendance management application.
- Store historical records using nested lists and dictionaries.
- Work with dates.
- Prevent duplicate records.
- Search and update stored data.
- Calculate percentages.
- Use indexing to access individual records.
- Use negative indexing to access the latest record.

- Use slicing to extract a portion of a list.
 - Understand the difference between indexing and slicing.
 - Build a structured Streamlit application.
 - Use forms, tabs, columns, sidebar, and session state.
 - Use visual formatting to communicate information clearly.
-

Main Lesson

Problem 16

Product

↓

Current Quantity

The focus was mainly on **current state**.

Problem 17

Student

↓

Attendance History

↓

Multiple Dates

↓

Multiple Records

And now:

Indexing → Access ONE record

Slicing → Access MULTIPLE records

This is the first problem where **Indexing & Slicing are explicit practical requirements**, not just topics listed at the end.

The next problems should build on these skills rather than repeating the same exercise.